CSCI 3308 Final Project: SpotDrop

Team: 013-4

Contributors

1. Brock Benton
2. Alex Jost
3. Atharva Joshi
4. Ganesh Akhil Meduri
5. Sam Kasten

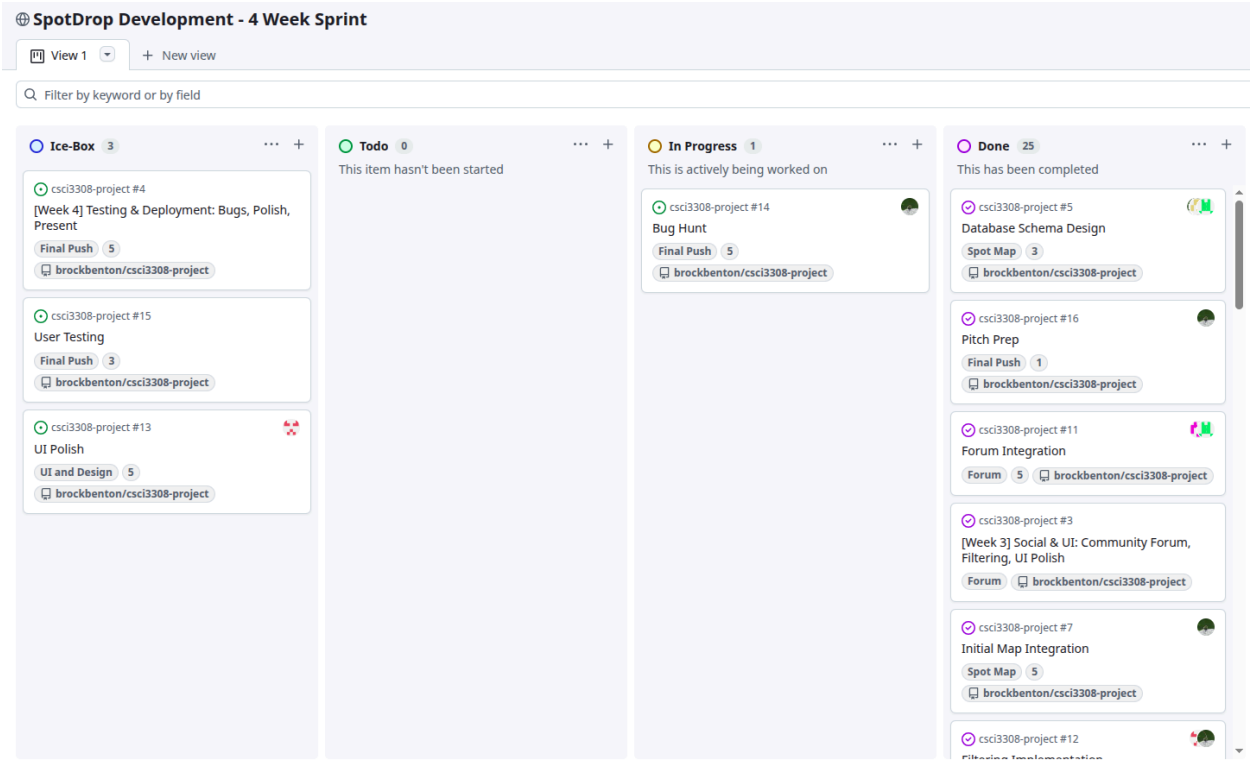
Project Description

SpotDrop is a community-driven web application built for urban athletes to discover and share sports spots including skate parks, climbing walls, parkour locations, and ski areas. The platform centers around an interactive map powered by Leaflet.js and OpenStreetMap, allowing users to visually browse spots filtered by sport type across a geographic area. Registered users can contribute new spots by dropping a pin, uploading photos, and adding descriptions with difficulty ratings. Each spot hosts its own discussion forum, enabling the community to comment, share tips, and engage with one another.

The application is built on a Node.js and Express.js backend with PostgreSQL as the database, using Handlebars for server-side templating and Bootstrap for responsive UI styling. Media uploads are handled through Cloudinary, and the app is containerized with Docker for consistent local development. Authentication is managed via bcryptjs and express-session, ensuring protected routes for spot creation and forum interaction. The project follows an MVC-style architecture with RESTful API endpoints and includes an automated test suite using Mocha and Chai. SpotDrop is deployed and publicly accessible on Render, seeded with real-world sports spots in the Denver area as sample data.

Project Tracker

Link: https://github.com/users/brockbenton/projects/2



Video Demo

Link: https://youtu.be/imrThMTG19U

Github Repository

Link: https://github.com/brockbenton/csci3308-project

Contributions

1. **Brock Benton:** Built the interactive Leaflet.js map, including map rendering, marker placement, and sport-type filtering. Developed the full add spot flow and the delete spot feature with proper authentication checks. Implemented the Find Me geolocation button, enabling users to center the map on their current location. Integrated Cloudinary media upload via Multer for spot photo submissions. Deployed the application to Render, configuring environment variables, the PostgreSQL instance, and the production build pipeline. Technologies used include Leaflet.js, Node.js/Express, PostgreSQL, Cloudinary, Multer, Render.

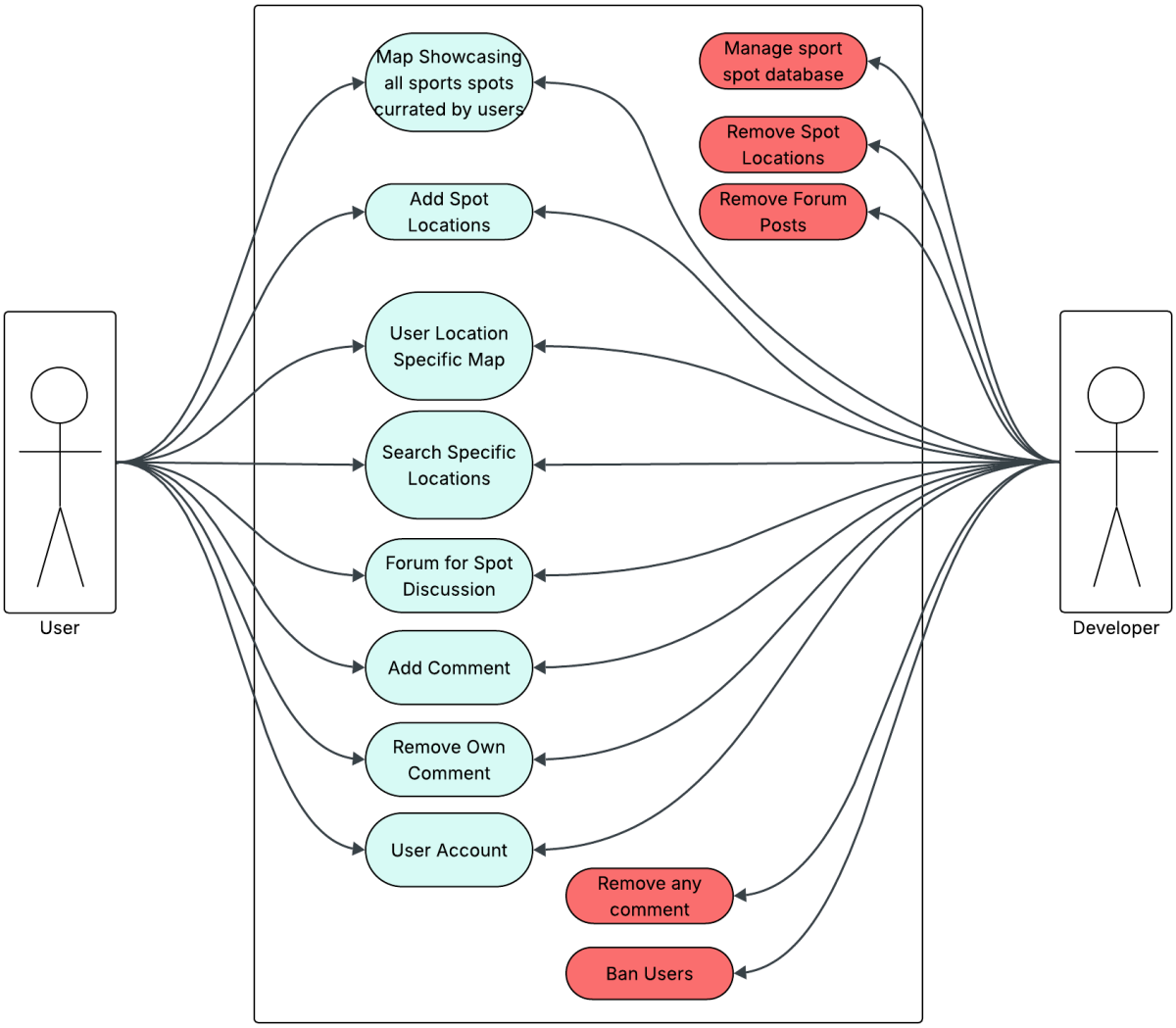
2. **Alex Jost:** Implemented the database schema for users. Developed backend and frontend of user login and registration, as well as the UI for both of these pages and the Account page. Added the functionality that allows users to change their profile picture based on set options available to them. Technologies used include PostgreSQL and Node.js/Express.js

3. **Atharva Joshi:** Built the Forums page UI. Helped with designing the database schema for forums. Implemented functionality for comments where the MTC timestamp is shown and formatted. Technologies used include Node.js/Express, PostgreSQL, Handlebars/HTML.

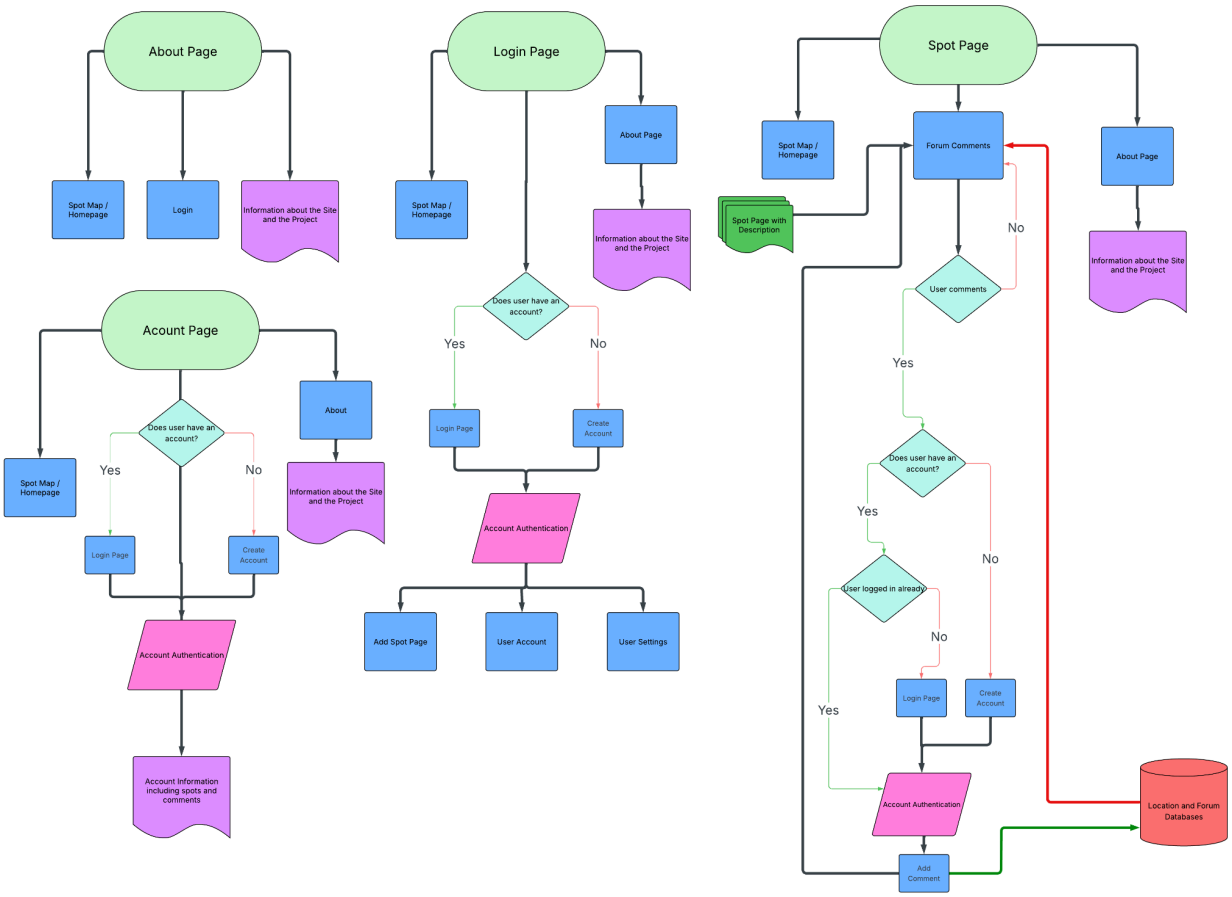
4. **Ganesh Akhil Meduri:** Designed and implemented the database schema for forums page. Developed the backend for forums page,made sure that each spot has its own discussion page, added the comment functionality where it shows all the comments within each forum and added the deleted functionality in forum page that made sure that only the users can delete their own comments. Technologies used PostgreSQL, Node.js,Express.js.

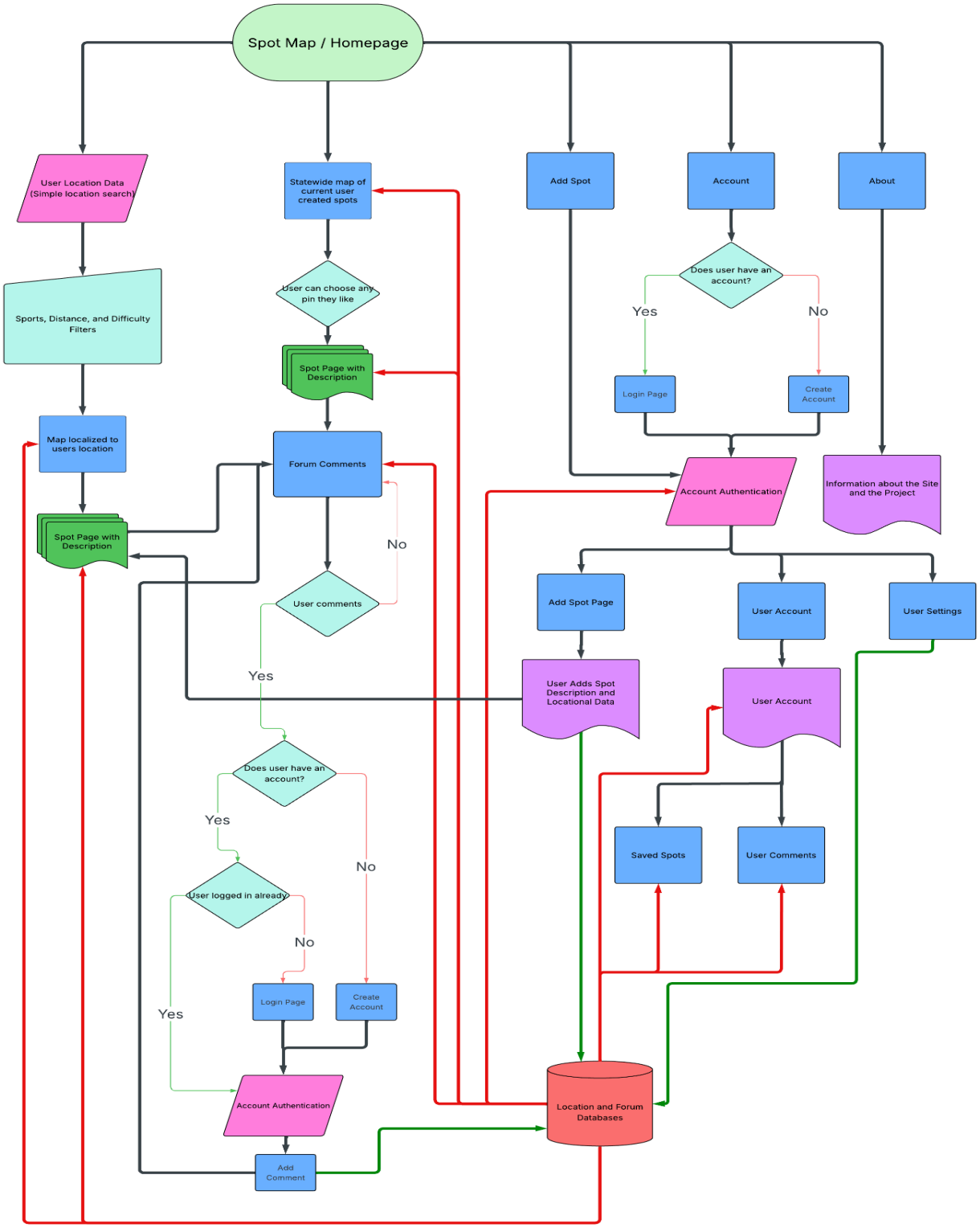
5. **Sam Kasten:** Designed the webpage UI/UX concept within Figma for quick prototyping. Developed and implemented multiple different logos and additional company imagery within Illustrator. Implemented dropdown feature on login button. Created the search location feature with integration from the Leaflet using data from the OpenStreetMaps database on cities and locations. Implemented a basic about page with clean aesthetics and information about the site. Updated UI/UX throughout the entire site including: Font size and color, moving buttons and comment boxes into flexboxes, added logos, customized buttons, connected photos to forums, creating scroll function for smaller devices, and ensured accessibility. Fixed a multitude of bugs including buttons and user checking. Technologies used include: Leaflet, OpenStreetMaps, Node.js/Express, PostgreSQL, Cloudinary, Illustrator, Figma, Lucidchart

Use Case Diagrams



Wireframes

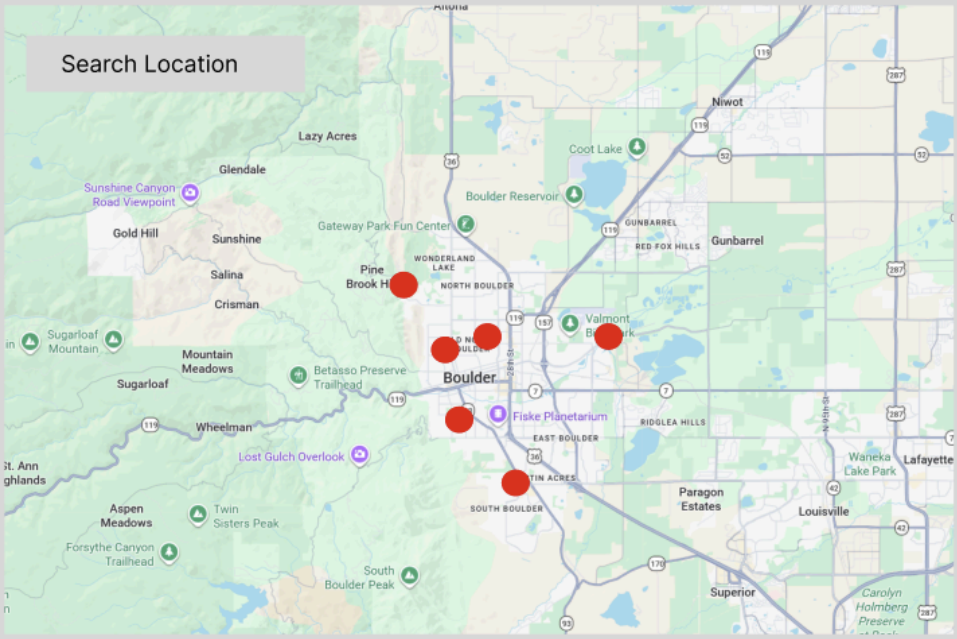

About Page
Spot Map / Homepage
Login
Information about the Site and the Project
Acount Page
Spot Map / Homepage
Does user have an account?
Yes
No
Login Page
Create Account
About
Information about the Site and the Project
Account Authentication
Account Information including spots and comments
Login Page
Spot Map / Homepage
About Page
Information about the Site and the Project
Does user have an account?
Yes
No
Login Page
Create Account
Account Authentication
Add Spot Page
User Account
User Settings
Spot Page
Spot Map / Homepage
Forum Comments
About Page
Spot Page with Description
Information about the Site and the Project
User comments
No
Yes
Does user have an account?
Yes
No
User logged in already
No
Yes
Login Page
Create Account
Account Authentication
Add Comment
Location and Forum Databases


Spot Map / Homepage
User Location Data (Simple location search)
Statewide map of current user created spots
Add Spot
Account
About
Sports, Distance, and Difficulty Filters
User can choose any pin they like
Does user have an account?
Yes
No
Spot Page with Description
Login Page
Create Account
Map localized to users location
Forum Comments
Account Authentication
Information about the Site and the Project
Spot Page with Description
No
User comments
Add Spot Page
User Account
User Settings
Yes
User Adds Spot Description and Locational Data
User Account
Does user have an account?
Yes
No
Saved Spots
User Comments
User logged in already
No
Yes
Login Page
Create Account
Account Authentication
Location and Forum Databases
Add Comment

Home Page

Add Spot Spot Map Account About

Search Location



Filter Sport

Filter Distance

Filter Rating

Spot

Add Spot Spot Map Account About

Spot Name:
Spot Location:
Spot Description:

Comments

Test Results

Link to full UAT Testing: UAT Plan

Create Account Test:

- TC 1.0 - User should be able to navigate to a create an account page and must provide the following mandatory fields to create an account before submitting and granted an account:
    - Username
    - Password
    - Email
- Actual Results: System worked as intended, users are able to create accounts using the mandatory fields and are prompted when they don't fully fill out the fields, users were able to understand the intent without being prompted. Users were also able to locate buttons and boxes as hoped.
- End User Comments: "It is easy to understand what I need to do" - Riley

Log In Test:

- TC 2.0 - Users should be able to navigate to a login page to be able to log into their account and given they provide the correct information into the mandatory fields they should be able to log in:
    - Username
    - Password
- Actual Results: System works as intended, users were able to log in to their accounts with the correct information, were prompted when they didn't have the right information, and were able to complete the task without help. Users were able to locate buttons and text boxes as hoped for.
- End User Comments: Again not much to comment on besides,
    - "yeah it worked" - Riley, and "Simple enough to me" - Chad

Navigate Maps Test:

- TC 4.0 - Users should be able to navigate the map with or without an account
    - Acceptance Criteria:
        - Users should be able to click and hold move the map to find spots
        - Users should be able to click spots to find more information on them
        - Users should be able to search different locations by typing different cities or states in the search bar
        - Site should attempt to predict the location while the user types a location into the search and provide suggestions
        - Map should move to the location when user searches a city or state
        - Users should be able to click a button to give their current location to see that location on the map

        - Site should prompt the user to consent to giving geolocation data, if user does the site should move the map to that location, if not map should just go back to default location
- Actual Results: This also worked as intended, but the end users did give good feedback on what they would like to be improved. Users were able to grasp quickly that the mouse was able to move around the map and they were free to click on the pins on said map. Worked as intended as the different color pins also prompted the users to explore and see what they were. They did wish for users to be able to share the spot with others so they could coordinate to said spot for meet ups. We unfortunately did not have enough time to be able to implement a share function.
- End User Comments: They liked the map and being able to select different pins and search different cities for spots. They did discuss they would like to be able to share the pin to others.
    - "Yeah the map is cool with the pins and stuff, being able to check out what people say about the spot is sweet, but how do I share it with a friend? I want to be able to give it to a buddy and meet up there" - Riley
    - "This is awesome this has some potential, we just need to get a lot more spots on this first, but then I could see it getting big" - Chad

Add Spot Test:

- TC 3.0 - Users with accounts should be able to add spots to the spot map
    - Acceptance Criteria:
        - Users must be logged into an account to be able to add spots.
        - Site should detect if a user is logged in when they want to add a spot, prompting them to sign in or create an account if they are not currently logged in.
        - Users cannot submit a spot without completing all the mandatory fields including:
            - Name
            - Sport Type
            - Difficulty
            - Description
            - Photo (Optional)
- Actual Results: System worked as intended, though users did give good feedback and what should be improved, They were able to find the add spot button easily and were quick to understand they simply need to press in the location they like to add the spot. They did wish for more photos for both the spot and the comments, which we unfortunately did not have enough time to implement. Also a slight issue with Iphone .heic photos, which apple uses for live photos, they do not work in the photo uploads as they do not work with our upload system.
- End User Comments: Having more than one photo per spot was something the end users brought up and wished we implemented, along with adding pictures in the comments for updates to a certain spot, lastly they want to be able to share the location a little easier by adding a share feature.

- “It would be better if you could add more photos to the spot, you know so people can see it at different angles, or if it's a park it could show different parts of the park.” - Riley
- “You should also make it easy to share the spot with other people you know so when we are out skating I can send the pin to my dudes to meet up” - Riley
- “You know what’d be sick too? If you could add photos to the comments, like if something broke or if a spot got skate stopped, people could show pictures of that in the comments.” - Chad

Deployment

Link: https://csci3308-project-x51h.onrender.com/

Our application was deployed via Render, using a managed PostgreSQL database instance and a web service connected to the GitHub repository. The web service was configured to automatically redeploy on pushes to the main branch. Environment variables (database credentials, session secret, and Cloudinary API keys for media uploads) were set directly in Render's dashboard rather than committed to the repo. The application runs on Node.js and connects to the Render-hosted PostgreSQL database, which was seeded with initial spot data. Cloudinary was used as the external media storage provider, with uploaded images stored remotely and referenced by URL in the database. Locally, the app can be run with Docker Compose, which spins up both the Node.js web server and a PostgreSQL 14 container, automatically seeding the database on first run.